

10장)_VisionTransformer

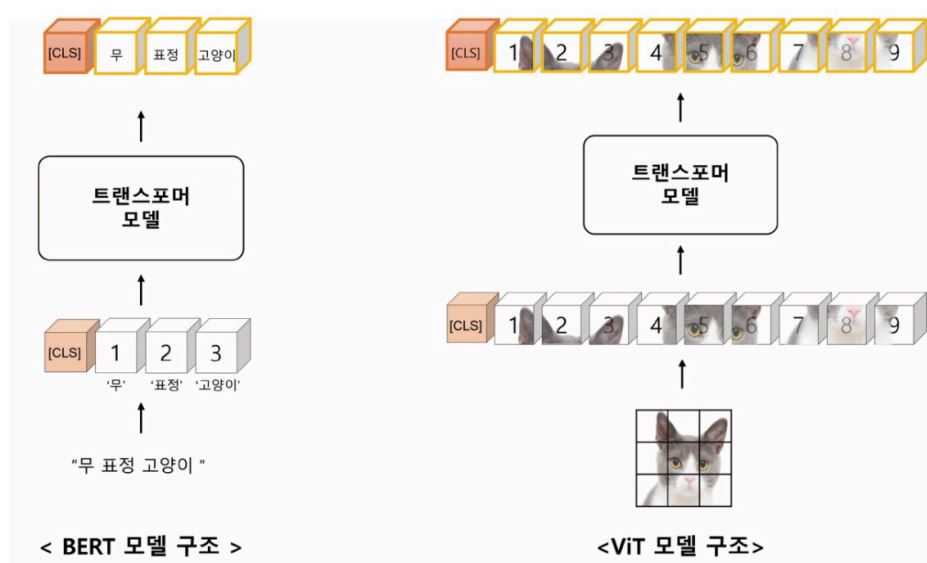
Vision Transformer

- 2020년 구글의 논문을 통해 소개된 이미지 인식을 위한 딥러닝 모델 → 이미지를 자연어 처리 방식처럼 분류해 보려는 시도에 의해 탄생
- 트랜스포머 모델에서 사용되는 셀프 어텐션 적용 → 셀프 어텐션을 사용해 전체 이미지를 한 번에 처리하는 방식으로 구현
 - 이미지 분류 모델에서 사용되는 CNN 모델의 합성곱 계층 방법(이미지 분류 위해 지역 특징을 추출)을 사용X
- 그러나, ViT 모델은 이미지 패치가 원 - 오, 위-아래 방향으로 순차적 입력 → 2차원 구조의 이미지 특성을 온전히 반영X
- 이 문제 해결 위해 물체의 크기나 해상도를 계층적으로 학습하는 Swin Transformer 와 CvT(Convolutional Vision Transformer) 모델이 제안됨
- **Swin Transformer** : Local Window를 활용해 각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 구성해 이미지의 특징을 학습 시킴
 - ViT 구조와 비교할 때 기존 셀프 어텐션을 로컬 윈도우 안에 대한 어텐션, 로컬 윈도우 간의 어텐션으로 수행하여 이미지 특징을 계층적으로 학습시킴 → 이 어텐션 함수에는 상대적 위치 편향을 반영하여 어텐션 값 자체에 위치적 정보를 포함시킴
- **CvT** : 기존 합성곱 연산 과정을 ViT에 적용한 모델
 - Low-level feature과 High-Level Feature를 계층적으로 반영할 수 있음
 - 어텐션 연산 과정에서 Query(Q), Key(K), Value(V) 중 Key와 Value를 기존 특징 벡터보다 축소해 계산 복잡도 감소시킴
- ViT 계열 모델들은 이미지 분류 작업에 매우 효과적인 모델 + 광범위한 다른 컴퓨터비전 작업에도 적용가능

ViT

- 자연어 처리 분야에서 트랜스포머 모델의 큰 성능 향상 → 컴퓨터비전 분야의 모델에도 영향
- ViT는 트랜스포머 구조 자체를 컴퓨터비전 분야에 적용한 첫번째 연구
 - [이전 CV분야] 합성곱 신경망에 트랜스포머 모델의 셀프 어텐션 모듈을 적용한 모델은 많았음

BERT 모델과 ViT 모델의 차이



- [공통점] both 트랜스포머 모델의 구조 그대로 사용
- [차이점] 입력 데이터를 만드는 과정
 - BERT : 문장보다 작은 단위의 토큰들이 순차적으로 입력
 - ViT : 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력
 - 입력 이미지 패치는 원 - 오, 위 - 아래로 표현된 시퀀셜 배열을 가정

합성곱 모델과 ViT 모델 비교

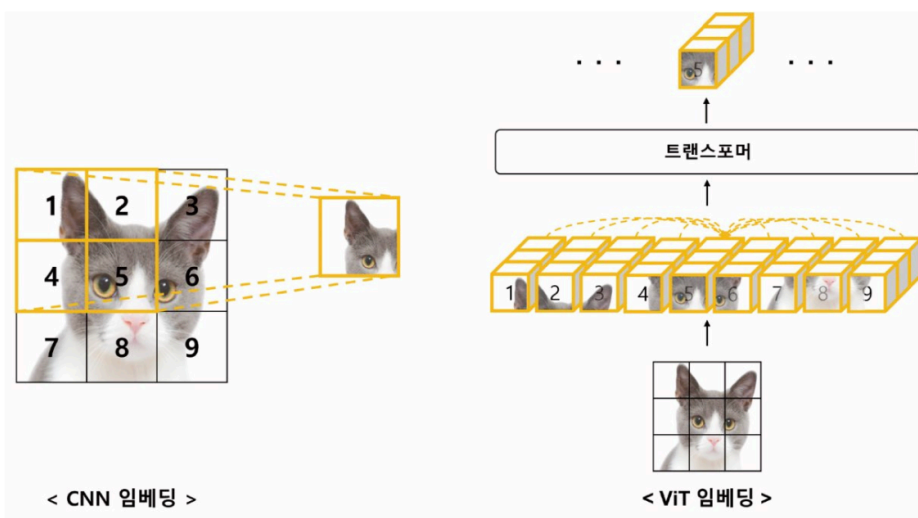


그림 10.3 합성곱 신경망과 ViT 계층의 차이

- [공통점] 이미지 특징을 잘 표현하는 임베딩을 만들고자 하는 공통의 목적을 가짐
- [차이점] 이미지 특징을 잘 표현하는 임베딩을 만드는 과정
 - CNN
 - 이미지 패치 중 일부만 선택하여 학습 → 이를 통해 이미지 전체의 특징 추출
 - 고양이의 이미지에서 왼쪽 눈의 특징을 추출하고자 한다면 CNN은 이미지에서 해당 부분만 선택 학습
 - 좁은 Receptive Field를 가짐 → 전체 이미지 정보를 표현하기 위해 많은 계층 필요
 - ViT
 - 이미지를 작은 패치들로 나눠 각 패치 간의 상관관계 학습
 - 셀프 어텐션 방법 사용해 모든 이미지 패치가 서로에게 주는 영향을 고려 → 이를 통해 전체 특징 추출
 - 모든 이미지 패치가 학습에 관여 + 높은 수준의 이미지 표현 제공
 - 어텐션 거리(Attention Distance)를 계산 → 오직 1개의 ViT Layer로 전체 이미지 정보 쉽게 표현

ViT의 귀납적 편향(inductive Bias)

- **귀납적 편향 : 일반화 성능 향상을 위한 모델의 가정(Assumption)**
 - 해당 모델이 가지는 구조와 매개변수들이 데이터에 적합한 가정을 하고 있음을 나타냄
 - 가정이 옳바르다면 모델은 더 적은 데이터로 높은 일반화 성능 보임
 - but, 너무 강한 귀납적 편향은 다른 유형의 데이터나 관계를 표현하는데 어려움 초래
- **이미지 데이터 - CNN - 지역적 편향**
 - 이미지 데이터는 공간적 관계를 잘 표현 → Local Bias 가진 CNN 모델을 많이 사용
 - 이미지를 작은 조각으로 나누어 각 조각의 특징 추출 → 조합하여 전체 이미지 특징 추론
 - 지역적 특성 가짐 → 이미지 데이터에서 좋은 성능 발휘
- **시계열 데이터 - RNN - 순차적 편향**
 - 시계열 데이터는 시간적 관계를 잘 표현 → Sequential Bias 가진 RNN 모델을 많이 사용
 - 이전 시간의 상태를 기억 + 현재 입력과 결합하여 다음 상태 예측
 - 순차적인 특징 강조하는 특성 가짐 → 시계열 데이터에서 좋은 성능 발휘
- but, 이런 특정 관계 편향이 강할 수록 다른 다양한 관계를 표현하기 어려움 = 귀납적 편향이 약한 모델 선호하게 됨
- **ViT는 귀납적 편향이 거의 없음**
 - 입력 데이터의 다양한 쿼리, 키, 값의 임베딩 형태로 일반화된 관계를 학습 → 귀납적 편향이 거의 없고 대용량 데이터에 대해서 이미 지 특징들의 관계를 잘 학습

ViT 모델

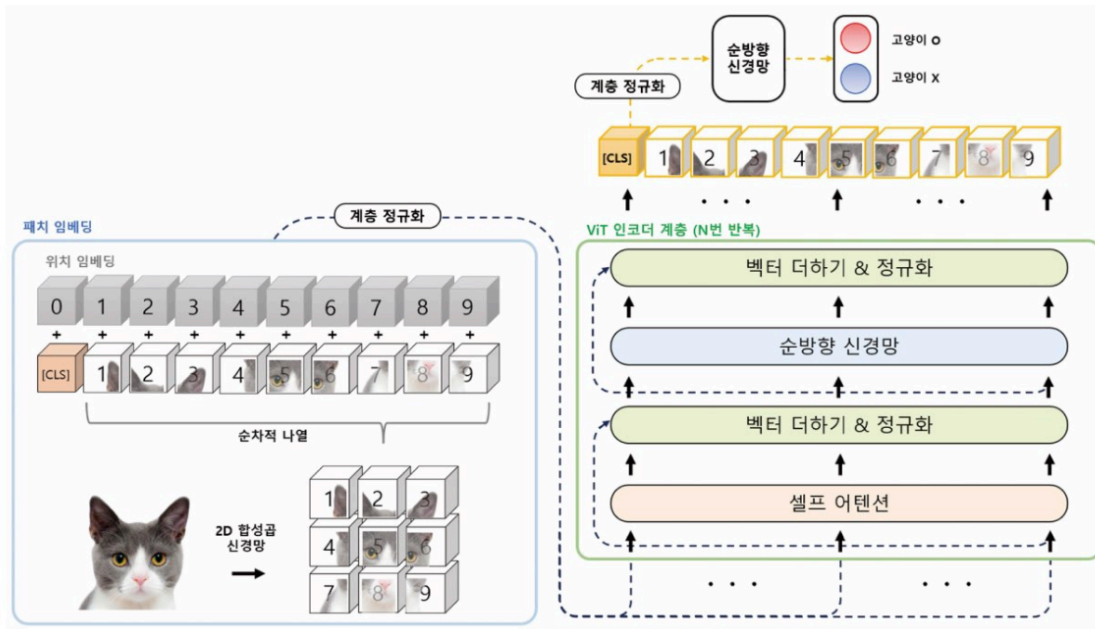


그림 10.4 ViT 모델 구조

구성 요소

1. 패치 임베딩(Patch Embedding) : 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기의 패치로 나눈 다음 각 패치를 벡터 형태로 변환
2. 인코더(Encoder) : 각 패치와의 관계를 학습하는 계층
 - 패치임베딩과 인코더 계층을 통해 이미지의 특징을 추출하고 분류나 회귀와 같은 작업에 맞는 출력값으로 변환해 사용

패치 임베딩(Patch Embedding)

- 입력 이미지를 작은 패치로 분할하는 과정
1. 작은 패치로 분할하기 위해 이미지 크기를 맞추는 전처리 수행 필요
 - 입력이미지의 크기가 640x480이었다면, 224x224크기와 같은 정방형 크기로 이미지 크기를 조절
 2. 이미지 크기를 일정한 크기로 변경했다면, 전체 이미지를 패치 크기로 분할해 시퀀셜 배열을 만듦
 - 이때, CNN 계층을 활용

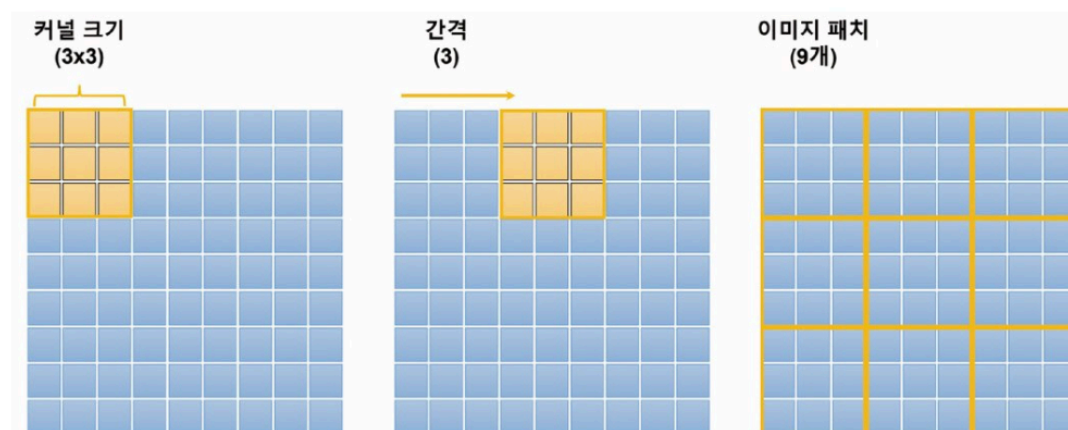


그림 10.5 이미지 패치 생성 과정

- Kernel Size와 Stride 설정 ← 하이퍼파라미터로 정의됨
 - Kernel : 패치의 크기, Stride : 패치가 이동하는 폭
- 패치 계산 과정

수식 10.1 패치 계산 과정

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9 - 3}{3} + 1 = 3$$

- 이미지 크기(9x9), Kernel Size(3x3), Stride(3) = 총 9(3x3) 이미지 패치 생성

3. 분할된 이미지 패치들을 배열 형태로 나열할 때, 왼-오, 위-아래 순서로 배열
4. 이 배열의 가장 왼쪽에 분류 토큰(Special Classification Token [CLS]) 추가
 - 전체 이미지를 대표하는 벡터로 특징 문제를 예측하는데 사용됨

5. 위치 임베딩(Position Embedding)을 사용하여 인접한 패치 간의 관계 학습

- 패치의 위치 정보를 임베딩 벡터로 변환하고 기존 이미지 패치 벡터들과 더함

6. 마지막으로 계층 정규화를 적용하면 패치 임베딩이 만들어짐

- 이렇게 이미지를 작은 패치로 나누어 처리하면 GPU 메모리의 한계를 극복하고 더 큰 이미지를 처리할 수 있음

인코더 계층(Encoder)

- ViT 모델은 이미지를 일정한 크기의 패치로 나눠 각 패치를 벡터로 변환한 후, 인코더 계층에 입력으로 전달 → 다양한 쿼리, 키, 값 임베딩의 관계를 학습
- N개의 인코더 레이어를 반복적으로 적용
- 마지막 레이어에서는 분류 토큰이라고 불리는 특별한 배치의 특징 벡터 추출
 - 분류 토큰 벡터 : 이미지 데이터를 잘 표현하는 특징 벡터 → 이후 다양한 이미지 분류 및 검색 문제 해결에 사용
 - (ex) 고양이 여부 구분 모델 학습 시, ViT 모델은 Sequential Output 모드를 사용하는 것이 아닌 전체 이미지의 특징을 잘 표현하는 분류 토큰 벡터만 사용하여 분류 문제 해결 → 따라서 분류 토큰 벡터를 RNN(or CNN)에 연결하고 고양이 여부를 분류하는 방법으로 ViT 모델이 학습됨

Swin Transformer

- 2021년에 발표한 대규모 비전 인식 모델로 아래와 같은 문제를 극복하기 위해 계층적(hierarchy) 특징 맵을 구성해 1차(Linear) 계산 복잡도를 갖는 스윈 트랜스포머 등장
 - [기존 Transformer 기반 모델의 공통적 문제점]
 - 고정된 배치 크기로 인해 세밀한 예측을 필요로 하는 의미론적 분할(Semantic Segmentation)과 같은 작업에 부적절
 - Transformer의 self-attention은 입력 이미지 크기에 대한 2차(Quadratic) 계산 복잡도를 가지므로 고해상도 이미지를 처리하기 어려움
- 이미지 분류, 객체 감지, 영상 분할과 같은 인식 작업에 강력한 성능 → Transformer 구조보다 이미지 특성을 더 잘 표현
- 시프트 윈도우(Shifted Window)라는 기술 이용해 입력 이미지를 일정한 크기의 패치로 분할 → 이 패치들을 쌓아 윈도우 구성 → 구성된 윈도우 영역만 셀프 어텐션 계산
 - 기존 트랜스포머 모델과 달리 패치를 겹쳐 더 큰 효율성 제공
 - 계층마다 어텐션이 수행되는 패치의 크기와 개수를 계층적으로 적용해 처리하기 때문에 → 높은 해상도와 다양한 크기 가진 객체를 효율적 처리 가능
- 이러한 특징을 바탕으로 객체 탐지, 객체 분할 등과 같은 작업에서 백본 모델로 사용됨

ViT와 스윈 트랜스포머 차이

1. ViT(Vision Transformer)

- 획일적인 패치 크기와 위치에 대한 제약 → 다양한 이미지 크기와 종횡비를 가진 데이터 세트에 대해서는 적용이 어려움
- 패치 단위로 입력을 받기 때문에 → 이미지 공간 정보가 완전히 보존되지 않을 수 있음
- 이미지 패치 수가 증가하면 학습 데이터의 크기도 커짐 → 모델 학습에 필요한 계산량 증가 → 대규모 데이터셋에서는 학습에 필요한 컴퓨팅 자원이 많이 사용됨

2. Swin Transformer

- Local Window 활용해 물체의 크기나 해상도를 계층적으로 학습
 - Local Window: : 스윈 트랜스포머에서 입력 이미지를 처리하기 위해 사용되는 고정 크기의 작은 윈도우
- Local Window를 이동시키는 Shift Window 기술을 도입해 입력 이미지를 일정한 크기의 패치로 분할하고 특징 맵을 이동시켜 다음 계층에 사용 → 패치의 위치를 더 유연하게 다룸

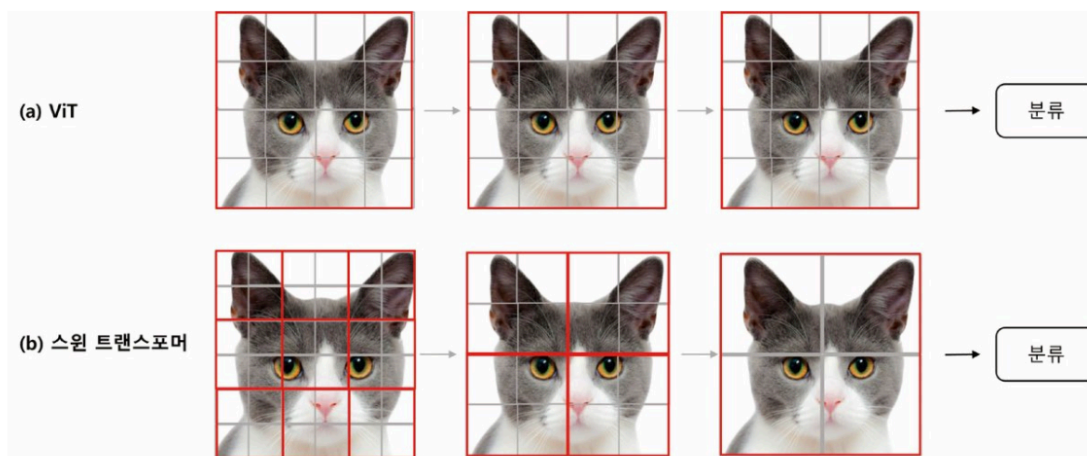


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

- **ViT의 네트워크** : 계층마다 전체 이미지 안에 어텐션이 수행되는 패치 크기와 개수가 동일
- **Swin Transformer의 네트워크** : Local Window를 통해 계층마다 어텐션이 되는 패치의 크기와 개수를 계층적으로 다양하게 적용
 - 입력 패치의 로컬 윈도우를 통해 계층적으로 접근
 - 시프트 윈도우로 로컬 윈도우를 이동해 가면서 인접 패치 간 정보 계산
 - 이런 계층적 접근 방식
 - 로컬 윈도우가 각 패치의 세부 정보에 집중하게 됨
 - 시프트 윈도우를 통해 전역 특징을 확인할 수 있음 → 고해상도 이미지를 효율적으로 처리

표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

스윈 트랜스포머 모델 구조

- 이미지를 패치 파티션으로 구분 후 → Linear Embedding(선형 임베딩), Swin Transformer Block(스윈 트랜스포머 블록), Patch Merging(패치 병합)연산이 반복적으로 수행됨
1. 먼저 입력 이미지를 일정한 크기의 패치로 분할 + 각 패치에 대해 선형 임베딩 수행 → 각 패치는 고정된 차원의 벡터로 변환됨
 2. 분할된 패치들을 기반으로 스윈 트랜스포머 블록 구성
 - 이 블록은 어텐션 계층, 계층 정규화, 다층 퍼셉트론 등을 포함하며 패치 간 상호작용을 수행
 3. 패치를 병합해 전체 이미지에 대한 분류 수행
 - 분할된 패치들을 순차적으로 합치면서 이미지의 전체적인 정보를 추출하는 방식 사용

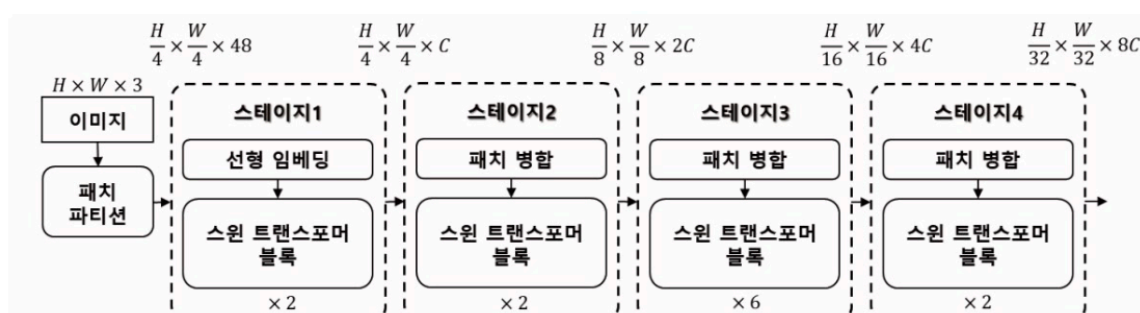


그림 10.7 스윈 트랜스포머 구조

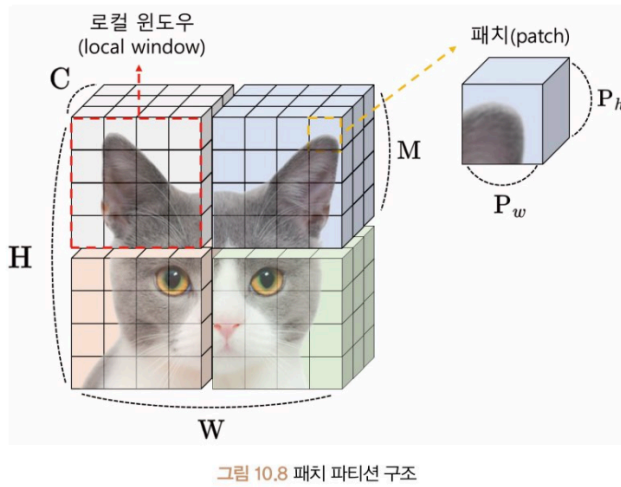
- 일반적으로 4개의 stage로 구성 → 각 stage는 서로 다른 해상도와 패치 크기를 가짐
 - Stage1 : 패치들이 선형 임베딩된 후 스윈 트랜스포머 블록에 전달됨
 - Stage 2, 3, 4 : 패치 병합 후 각 스윈 트랜스포머 블록에 전달됨
 - 패치 병합은 모델의 매개변수를 줄이기 위해 이미지 텐서 $[C, H, W]$ 를 $[2C, H/2, W/2]$ 로 재정렬해 저차원 임베딩을 수행

- Stage 1, 2, 3, 4에 공통적으로 사용되는 스윈 트랜스포머 블록은 윈도우 격자 안에 있는 패치 내부의 관계, 윈도우 격자 간의 관계를 학습하는 두 가지 어텐션 모듈로 이뤄져 있음

패치 파티션/ 패치 병합

1. Patch Partition : 입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식

- 분할된 패치는 트랜스포머 계층의 입력으로 사용됨 → 입력 이미지의 공간 정보를 보존하고 트랜스포머 계층에 대한 계산 효율성을 높임



- 일반적인 이미지 텐서는 $[C, H, W]$ 의 구조를 가짐 → 이 구조에서 패치 파티션을 통해 로컬 윈도우를 추가
 - M : 로컬 윈도우의 크기
 - 이미지 텐서의 차원이 $[3, 32, 32]$ 이라면 패치의 가로(P_h)와 세로(P_w)는 4가 됨

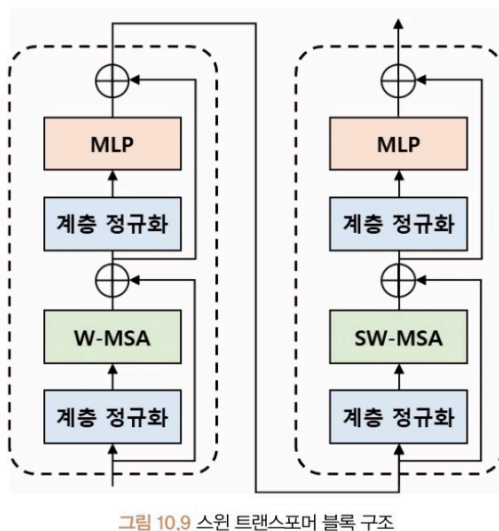
2. Patch Merging : 인접한 패치들의 정보를 저차원으로 축소하는 과정

- 모델의 매개변수를 줄이기 위해 이미지 패치 텐서 $[C, H, W]$ 를 $[2C, H/2, W/2]$ 로 재정렬해 $2C$ 의 저차원 임베딩을 수행
- (ex) 채널(C) 3개, 이미지 패치($H \times W$) 64개로 구성된 텐서 $[3, 8, 8]$ 가정 → 정의된 $[2C, H/2, W/2]$ 에 따라 $[6, 4, 4]$ 로 재정렬 → 증가된 6차원 채널을 원래 3차원으로 임베딩해 산출물 $[3, 4, 4]$ 텐서를 생성

스윈 트랜스포머 블록

- 패치 파티션 이후 네 개의 스테이지 안에서 반복 적용됨
- 선형 임베딩 도는 패치 병합 이후에 수행됨
- 계층 정규화, W-MSA(Window Multi-head Self-Attention), MLP, SW-MSA(Shifted Window Multi-head Self-Attention)등으로 구성

스윈 트랜스포머 블록 구조



- 트랜스포머 모델에서 사용되던 MSA 방식이 아닌 W-MSA와 SW-MSA로 구성되며 순차적으로 수행됨
- **W-MSA**

- 로컬 윈도우를 사용하는 멀티 헤드 셀프 어텐션 → 입력 특징 맵을 중첩되지 않게 나눈 다음 각 윈도우에서 독립적으로 셀프 어텐션 수행
- 이미지에서 일정한 크기의 윈도우 영역 설정 → 해당 영역 내 픽셀 간의 어텐션을 계산
- 이를 통해 이미지 내의 전체적인 어텐션 수행하고 특정 위치에 대한 정보 추출
- 각 윈도우 내 지역적인 특징 분석 가능하고, 전체 입력에 대한 셀프 어텐션 비용을 2차 계산 복잡도에서 1차 계산 복잡도로 줄임
- [장점] 연산량을 대폭 감소 ↔ [단점] 윈도우 내부의 이미지 패치 영역에만 셀프 어텐션을 수행

○ SW-MSA

- W-MSA의 단점을 보완
- 윈도우 사이의 연결성을 구축해 윈도우 간의 관계성 정보를 추출
- 윈도우를 가로 또는 세로 방향으로 이동한 후, 인접한 윈도우 간의 셀프 어텐션을 계산하는 방식

● W-MSA와 SW-MSA 어텐션 영역 비교

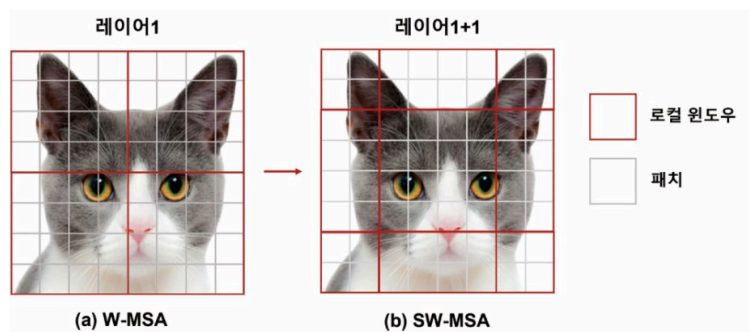


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

○ W-MSA

- 로컬 윈도우 안에서 패치 간의 셀프 어텐션 연산을 수행하는 방식
- 이때 로컬 윈도우가 이동하면서 각각의 윈도우에서 연산을 수행해야 하므로 계산량이 많아지는 것처럼 보일 수 있음
- But, **효율적인 배치 계산(Efficient Batch Computation)**을 통해 연산량 대폭 감소 → 로컬 윈도우에서 연산을 수행하는 과정에서도 배치 축을 사용할 수 있어서 연산 속도가 빨라지게 됨 → 자연어 처리 작업에서 높은 성능을 보일 수 있게 됨

○ SW-MSA

- 로컬 윈도우 간의 셀프 어텐션을 수행하기 위해 W-MSA에서 사용된 로컬 윈도우 개수보다 많아 더 많은 셀프 어텐션 연산을 요구
- 이 문제를 해결하기 위해 **순환 시프트(Cyclic Shift)**와 **어텐션 마스크(Attention Mask)** 사용

● Cyclic Shift를 활용한 SW-MSA 어텐션 계산 과정

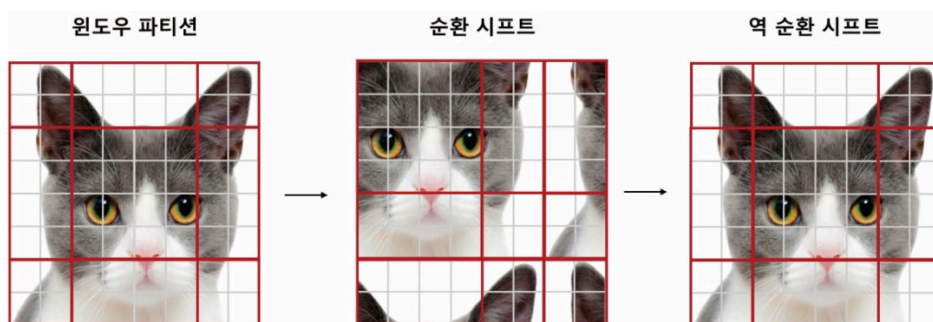


그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

- 순환 시프트는 로컬 윈도우가 이동할 때 이동한 위치의 정보를 이전 위치에서 가져오는 것을 의미
- 로컬 윈도우 사이즈 M 보다 작은 $M/2$ 만큼 로컬 윈도우들을 이동시킴
- 이때, 이동된 위치에서 연산을 수행하지 않도록 어텐션 마스크를 사용해 방지
 - 어텐션 마스크 : 로컬 윈도우가 이동한 위치에서 연산을 수행하지 않게 하는 역할
- 마지막으로 순환 시프트 단계에서 셀프 어텐션을 수행한 다음 **역순환 시프트(Reverse Cyclic Shift)**를 사용해 원래 로컬 윈도우 구조로 복원

⇒ 이러한 방식을 통해 SW-MSA 방식에서 윈도우 간에 셀프 어텐션 연산을 효율적으로 수행

- 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

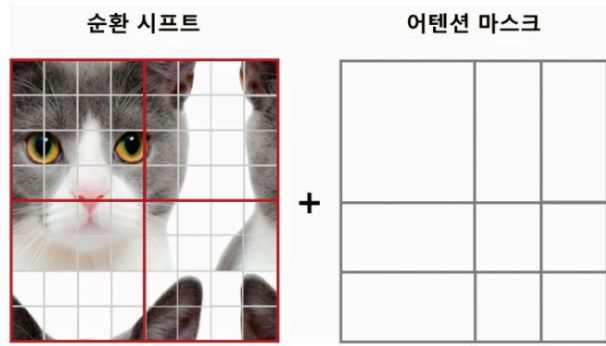


그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

- SW-MSA 방식은 순환 시프트와 함께 어텐션 마스크를 사용하여 로컬 윈도우 간의 셀프 어텐션 연산 효율적 수행
- 이때, (어텐션 마스크를 사용하면 원래 총 9개의 윈도우 파티션에 대해 9번의 셀프 어텐션 값을 구해야 했지만)
- 4개의 윈도우 파티션에 대한 4개의 셀프 어텐션 값만으로 모든 로컬 윈도우 간의 셀프 어텐션 값을 계산 가능
- 이는 어텐션 마스크가 순환 시프트를 통해 이동된 윈도우에 대한 정보를 보존 + 불필요한 연산을 줄이고 효율적인 계산을 가능하게 함

- 상대적 위치 편향 계산 과정



그림 10.13 상대적 위치 편향 계산 과정

- 스윈 트랜스포머에서 사용한 셀프 어션은 상대적 위치 편향 (Relative Position Bias) 고려
- 상대적 위치 편향은 로컬 윈도우 안에 있는 패치 간의 상대적인 거리를 임베딩하는 목적을 가짐
- 이미지 패치들이 1부터 4까지 주어진다면 패치 간의 상대적인 거리는 X축과 Y축 두 가지 방식으로 표기
 - X축 방식: 두 패치 간의 가로 방향 거리, Y축 방식: 두 패치 간의 세로 방향 거리
- X축에 대한 위치 행렬: 같은 X축에 놓여 있으면 상대적 거리 0, 아래로 떨어져 있다면 1, 위로 떨어져 있으면 1
- Y축에 대한 위치 행렬: 같은 Y축에 놓여 있으면 상대적 거리 0, 오른쪽으로 떨어져 있으면 -1, 왼쪽으로 떨어져 있으면 1

- 상대적 위치 편향 계산

```
import torch

window_size = 2
coords_h = torch.arange(window_size)
coords_w = torch.arange(window_size)
coords = torch.stack(torch.meshgrid([coords_h, coords_w], indexing="ij"))
coords_flatten = torch.flatten(coords, 1)
relative_coords = coords_flatten[:, :, None] - coords_flatten[:, None, :]

print(relative_coords) # 1, 2번 연산 과정
print(relative_coords.shape)
```


출력 결과

```
tensor([[[ 0,  0, -1, -1],
          [ 0,  0, -1, -1],
          [ 1,  1,  0,  0],
          [ 1,  1,  0,  0]],

        [[ 0, -1,  0, -1],
          [ 1,  0,  1,  0],
          [ 0, -1,  0, -1],
          [ 1,  0,  1,  0]]])
torch.Size([2, 4, 4])
```

- `window_size` : 여러 개의 이미지 패치를 포함하는 격자의 크기를 의미하며 윈도우 내외부의 패치를 구분해 주는 역할
- `torch.meshgrid` 함수: `coords_h` 배열 값(i)과 `coords_w` 배열 값(j)으로 사각형격자(ij)를 만드는 데 사용됨
 - 사각형의 X좌표. Y좌표에 해당하는 배열을 반환
- `torch.stack` 을 사용 → (X, Y)쌍 배열을 만든 후 → `torch.flatten` 으로 각 X, Y축에 대한 위치 인덱스 (`coords_flatten`) 생성
- 각 위치 색인의 차이를 계산하면 X, Y에 대한 위치 행렬 1, 2의 결과 얻을 수 있음

• X, Y축에 대한 위치 행렬

```
x_coords = relative_coords[0, :, :]
y_coords = relative_coords[1, :, :]

x_coords = x_coords * window_size - 1 #X축에 대한 3번 연산 과정
y_coords = y_coords * window_size - 1 #Y축에 대한 3번 연산 과정
x_coords = x_coords * 2 * window_size - 1 #4번 연산 과정

print(f"X축에 대한 행렬: \n{x_coords}\n")
print(f"Y축에 대한 행렬: \n{y_coords}\n")

relative_position_index = x_coords + y_coords #5번 연산 과정
print(f"X, Y축에 대한 위치 행렬:\n{relative_position_index}")
```

- `relative_coords` 를 각 X, Y좌표에 대한 `x_coords, y_coords` 로 분리
- 각 `x_coords, y_coords` 에 대해서는 `window_size - 1` 만큼 더해줌(3번)
- 그다음 `x_coords` 에 대해서 `2 * window_size - 1` 만큼 곱해줌 (4번)
- 마지막으로 `x_coords` 와 `y_coords` 를 더해주면 X, Y축에 대한 상대적 위치 좌표 행렬을 생성(5번)

• X, Y축에 대한 상대적 위치 좌표 변환

```
num_heads = 1
relative_position_bias_table = torch.Tensor(
    torch.zeros ((2 * window_size - 1) * (2 * window_size - 1), num_heads)
)

relative_position_bias = relative_position_bias_table[relative_position_index.view(-1)]
relative_position_bias = relative_position_bias.view(
    window_size * window_size, window_size * window_size, -1
)
print(relative_position_bias.shape)
```

출력 결과

```
torch.Size([1, 4, 4])
```

- `num_heads`: MSA를 계산할 때 사용된 헤드 수 → 예제에서 1 개로 설정

- relative_position_bias_table: 헤드별로 0~8번까지 9개의 상대적 위치 편향을 가지는 테이블
- relative_position_index를 이용해 해당 위치의 편향 값을 불러옴
- 이 값들은 view 함수를 이용해 헤드별 i번째 패치에서 j번째 패치의 어텐션 값에 더해지는 텐서 [4, 4, 1]로 재구성됨
- 이렇게 생성된 상대적 위치 임베딩(B)을 기존 트랜스포머에 사용되는 셀프 어텐션 수식에 적용

수식 10.2 상대적 위치 편향을 고려한 셀프 어텐션

$$Attention(Q, K, V) = SoftMax(QK^T / \sqrt{d} + B)V$$

CvT

- CVT(Convolutional Vision Transformer) : CNN구조에서 활용하는 계층형 구조(Hierarchical Architecture)를 ViT 에 적용한 모델
 - ViT의 셀프 어텐션은 이미지 정보를 바탕으로 각 패치 간의 관계를 학습
 - But, 모든 패치에 대해 동일한 크기와 간격을 사용 → 이미지의 물체 크기나 해상도를 계층적으로 학습하기가 어려움
- 이 문제를 해결하기 위해
 - 스윈 트랜스포머는 트랜스포머 블록마다 다른 로컬 윈도우 크기를 사용해 이미지의 계층적 특징을 학습
 - **CvT: CNN에서 사용하는 계층적 구조를 ViT에 적용 → 이미지의 지역 특징 (Local Feature)과 전역 특징(Global Feature)을 모두 활용해 비교적 적은 수의 매개변수로 높은 성능을 보임**
 - 이러한 CVT의 구조는 합성곱 계층의 지역 특징을 추출
 - ViT의 셀프 어텐션 계층으로 전역 특징을 결합해 이미지를 처리
 - 그러므로 CvT 모델은 스윈 트랜스포머보다 더 적은 수의 매개변수로도 비슷한 수준의 성능을 보임

• CvT 모델의 계층적 특징 구조 예시

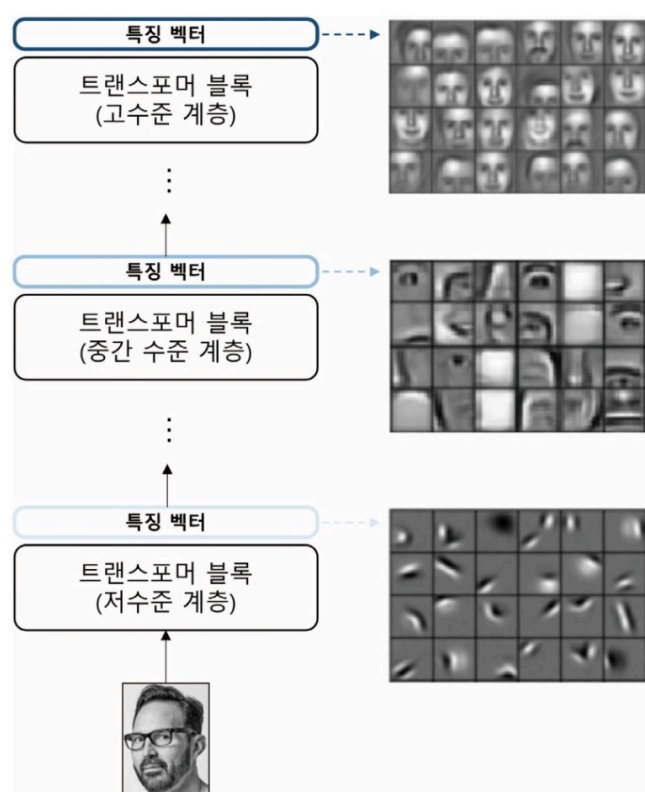


그림 10.14 CvT 모델의 계층적 특징 구조 예시

- 얼굴 이미지가 CvT 모델의 입력을 사용된다면
 - 저수준 계층에서 선 또는 점에 대한 특징 표현
 - 중간 수준에서는 눈이나 귀와 같은 특징을 표현
 - 고수준 계층에서는 얼굴 구조에 대한 특징 표현
 - 따라서, CvT의 계층형 구조는 물체의 크기나 해상도를 더 잘 표현함
- CvT는 이미지를 처리하기 전에 CNN을 통해 특징 맵을 추출 → 이를 통해 유의미한 패턴을 추출하고 일정한 크기의 패치로 나눠 트랜스포머 모델의 입력을 사용 → 입력되는 패치들은 겹쳐 학습
- (ViT는 패치 임베딩과 선형 임베딩을 사용하지만) CvT 모델은 합성곱 연산을 통해 이미지를 학습 → 이러한 방법을 통해 CvT 모델은 토근에 대한 위치 임베딩을 적용하지 않아도 모델의 성능 저하 없이 다양한 해상도를 가진 이미지 처리 할 수 있게 됨

표 10.4 ViT와 CvT 모델 비교

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

합성곱 토큰 임베딩

- Convolutional Token Embedding : 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩
- ViT 에 계층적인 합성곱 신경망의 성질을 추가하기 위해 사용
- 기존 트랜스포머 모델
 - 이미지의 특징을 추출하기 위해 9개의 이미지 패치를 사용
 - 선형 임베딩을 통해 쿼리, 키, 값 임베딩으로 변환
 - 임베딩을 사용해 셀프 어텐션을 계산하고 출력을 생성하므로 이미지의 근본적인 2D 모양의 구조를 잃게 됨
- 반면에 CVT 모델
 - 이미지의 2D 구조를 보존하면서도 셀프 어텐션을 계산하는 방식을 사용
 - 이미지 패치를 2D 합성곱 임베딩(2D Convolution Embedding)을 사용해 쿼리, 키, 값 임베딩으로 변환
 - 이렇게 변환된 임베딩을 사용해 셀프 어텐션을 계산하고 출력을 생성

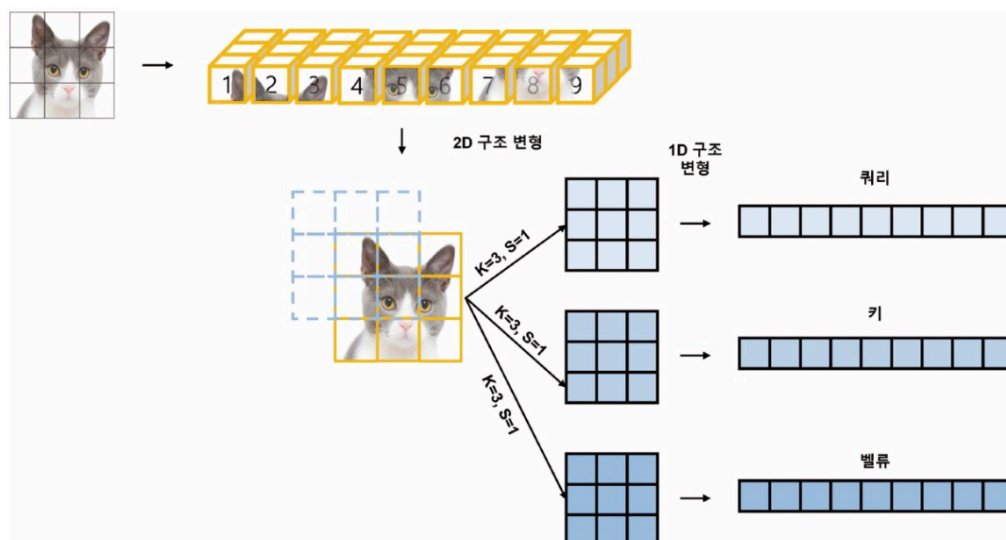


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

- 이미지에 3x3 커널 크기(K), 1간격(S)의 커널을 적용하면 패치 크기는 3x3으로 생성
- 각각의 패치를 쿼리, 키, 값에 대해 합성곱 연산을 적용 → 1차원 구조로 재배열해 기존 셀프 어텐션 방법으로 학습하는 방식
- 이를 통해 CVT 모델은 지역 특징을 학습할 수 있고, 시퀀스 길이를 줄이는 동시에 토큰 특징의 차원을 증가시킴
- 합성곱 모델에서 수행되는 방식처럼 다운 샘플링과 특징 맵의 수를 늘리게 됨

어텐션에 대한 합성곱 임베딩

- Convolutional Projection for Attention : 2D로 재구성된 토큰 맵에서 분리 가능한 합성곱 연산을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산
- (일반적으로 쿼리, 키, 값 임베딩은 차원을 동일하게 설정하지만)CvT 모델은 매개변수의 수를 줄이기 위해 쿼리, 키, 값의 차원을 축소시킴

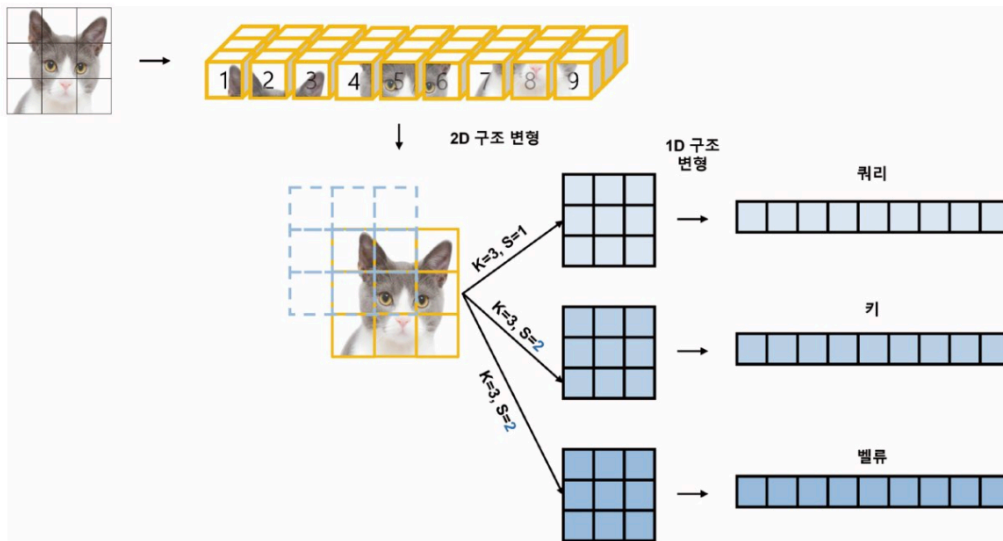


그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩

- 쿼리는 3X3 커널 크기(K), 1간격(S)의 커널을 적용해 9개의 이미지 패치를 생성
- 키와 값은 3X3 커널 크기(K), 2간격(S)으로 커널을 적용해 4개의 이미지 패치를 생성
- 길이가 다른 쿼리(9), 키(4), 값(4)을 사용하더라도 기존 셀프 어텐션의 결과값 사이즈가 9x1로 동일하게 출력될 수 있음

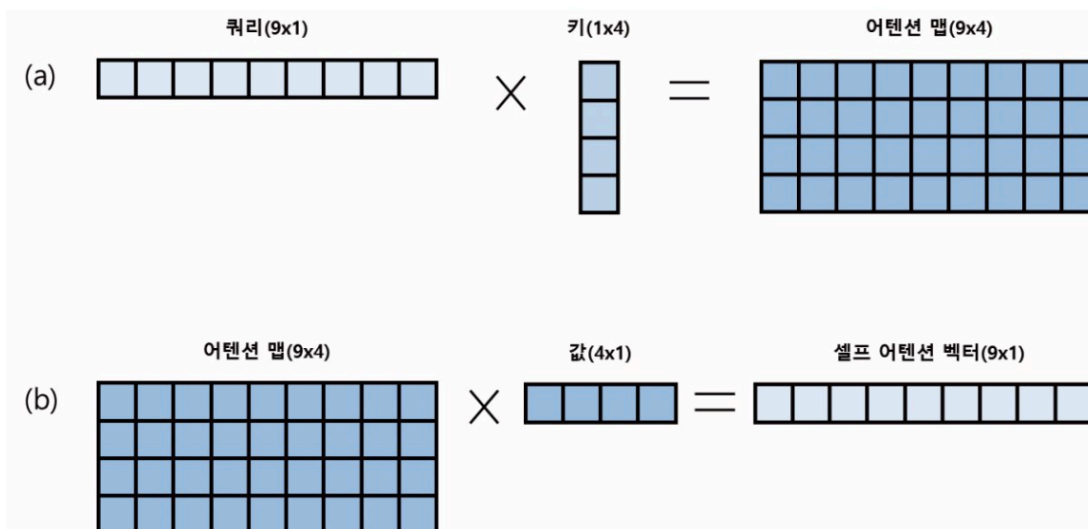


그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

- (a) : 9개의 쿼리 패치 임베딩과 4개의 키 패치 임베딩에 대한 중요도를 의미하는 어텐션 맵 생성
- 어텐션 맵이 생성되면 (b)와 같이 값 패치 임베딩으로 쿼리 어텐션 맵에 대한 가중치를 반영
 - 셀프 어텐션 벡터를 확인해 보면 입력 벡터와 출력 벡터의 패치 길이가 동일한 것 확인 가능
 - 따라서 계산해야 하는 이미지 패치 개수가 줄어들어 연산량 감소
- CvT 모델은 트랜스포머 계층이 깊어질수록 패치 간의 관계를 학습하는 길이를 축소 ↔ 대신에 벡터의 차원을 증가시켜 모델의 매개변수 수를 대폭 감소시킴
 - (ex) 첫 번째 트랜스포머 블록의 이미지 패치 크기가 64X64, 임베딩 차원이 128, 마지막 트랜스포머 블록의 이미지 패치 크기를 4배 감소시켜 8X8로 설정했다면, 기존 임베딩 차원도 4배 증가시켜 512차원으로 증가시킴
 - = 이미지 패치 크기가 줄어든 만큼 임베딩 차원을 늘리는 방법 → 네트워크의 안정성을 향상 + 모델 설계 단순화